

Optical Character Recognition Errors and Their Effects on Natural Language Processing

Daniel Lopresti
Department of Computer Science and Engineering
Lehigh University
19 Memorial Drive West
Bethlehem, PA 18015, USA
lopresti@cse.lehigh.edu

ABSTRACT

Errors are unavoidable in advanced computer vision applications such as optical character recognition, and the noise induced by these errors presents a serious challenge to downstream processes that attempt to make use of such data. In this paper, we apply a new paradigm we have proposed for measuring the impact of recognition errors on the stages of a standard text analysis pipeline: sentence boundary detection, tokenization, and part-of-speech tagging. Our methodology formulates error classification as an optimization problem solvable using a hierarchical dynamic programming approach. Errors and their cascading effects are isolated and analyzed as they travel through the pipeline. We present experimental results based on a large collection of scanned pages to study the varying impact depending on the nature of the error and the character(s) involved. The problem of identifying tabular structures that should not be parsed as sentential text is also discussed.

Categories and Subject Descriptors

I.7.5 [Document and Text Processing]: Document Capture—*document analysis*

General Terms

algorithms, measurement, performance

Keywords

performance evaluation, optical character recognition, sentence boundary detection, tokenization, part-of-speech tagging

1. INTRODUCTION

Despite decades of research and the existence of established commercial products, the output from optical character recognition (OCR) processes often contain errors. Since

such systems can form the first stage in a pipeline where later stages are designed to support sophisticated information extraction and exploitation applications, it is important to understand the effects of recognition errors on downstream text analysis routines. Are all recognition errors equal in impact, or are some worse than others? Can the performance of each stage be optimized in isolation, or must the end-to-end system be considered? What are the most serious forms of degradation a page can suffer in the context of natural language processing? In balancing the tradeoff between the risk of over- and under-segmenting characters during OCR, where should the line be drawn to maximize overall performance? The answers to these questions should influence the way we design and build document analysis systems.

Researchers have already begun studying problems relating to processing text data from noisy sources. To date, this work has focused predominately on errors that arise during speech recognition. For example, Palmer and Ostendorf describe an approach for improving named entity extraction by explicitly modeling speech recognition errors through the use of statistics annotated with confidence scores [15]. The inaugural *Workshop on Analytics for Noisy Unstructured Text Data* [20] featured papers examining the problem of noise from a variety of perspectives, with most emphasizing issues that are inherent in written and spoken language.

There has been less work, however, in the case of noise induced by optical character recognition. Early papers by Taghva, Borsack, and Condit show that moderate error rates have little impact on the effectiveness of traditional information retrieval measures [18], but this conclusion is tied to certain assumptions about the IR model (“bag of words”), the OCR error rate (not too high), and the length of the documents (not too short). Miller, *et al.* study the performance of named entity extraction under a variety of scenarios involving both ASR and OCR output [14], although speech is their primary interest. They found that by training their system on both clean and noisy input material, performance degraded linearly as a function of word error rates.

Farooq and Al-Onaizan proposed an approach for improving the output of machine translation when presented with OCR’ed input by modeling the error correction process itself as a translation problem [4].

A paper by Jing, Lopresti, and Shih studied the problem of summarizing textual documents that had undergone optical character recognition and hence suffered from typical OCR errors [8]. From the standpoint of performance evaluation, this work employed a variety of indirect measures:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

AND '08, July 24, 2008 Singapore

Copyright 2008 ACM 978-1-60558-196-5 ...\$5.00.

for example, comparing the total number of sentences returned by sentence boundary detection for clean and noisy versions of the same input text, or counting the number of incomplete parse trees generated by a part-of-speech tagger.

In two later papers [10, 11], we turned to the question of performance evaluation for text analysis pipelines, proposing a paradigm based the hierarchical application of approximate string matching techniques. This flexible yet mathematically rigorous approach both quantifies the performance of a given processing stage as well as identifies explicitly the errors it has made. Also presented were the results of pilot studies where small sets of documents (tens of pages) were OCR’ed and then piped through standard routines for sentence boundary detection, tokenization, and part-of-speech tagging, demonstrating the utility of the approach.

In the present paper, we employ this same evaluation paradigm, but using a much larger and more realistic dataset totaling over 3,000 scanned pages which we are also making available to the community to foster work in this area [12]. We study the impact of several real-world degradations on optical character recognition and the NLP processes that follow it, and plot later-stage performance as a function of the input OCR accuracy. In addition, the problem of identifying tabular structures that should not be parsed as sentential text is also discussed. We conclude by outlining possible topics for future research.

2. STAGES IN TEXT ANALYSIS

In this section, we describe the prototypical stages that are common to many text analysis systems, and then list the specific packages we use in our work. The stages, in order, are: (1) optical character recognition, (2) sentence boundary detection, (3) tokenization, and (4) part-of-speech tagging. These basic procedures are of interest because they form the basis for more sophisticated natural language applications, including named entity identification and extraction, topic detection and clustering, and summarization.

In selecting implementations of the above stages, we choose to employ freely available open source software rather than proprietary, commercial packages. From the standpoint of our work, we require behavior that is representative, not necessarily “best-in-class.” For sufficiently noisy inputs, the same methodology and conclusions are likely to apply no matter what algorithm is used. Comparing different techniques for realizing a given stage to determine which is most robust in the presence of OCR errors would make an interesting topic for future research.

2.1 Optical Character Recognition

The first stage of the pipeline is optical character recognition, the conversion of the scanned input image from bitmap format to encoded text. Optical character recognition performs quite well on clean inputs in a known font. It rapidly deteriorates in the case of degraded documents, complex layouts, and/or unusual fonts. In certain situations, OCR will introduce many errors involving punctuation characters, which has an impact on later-stage processing.

For our OCR stage, we selected the open source *Tesseract* package [19]. The latest version is 2.03. Since we are presenting it with relatively simple text layouts, having to contend with complex documents is not a concern in our experiments. The performance of *Tesseract* on the inputs we tested is likely to be similar to the performance of a

Dean Foods Co expects earnings for the fourth quarter ending May 30 to exceed those of the same year-ago period, Chairman Kenneth Douglas told analysts. In the fiscal 1986 fourth quarter the food processor reported earnings of 40 cts a share. Douglas also said the year’s sales should exceed 1.4 billion dlrs, up from 1.27 billion dlrs the prior year. He repeated an earlier projection that third-quarter earnings “will probably be off slightly” from last year’s 40 cts a share, falling in the range of 34 cts to 36 cts a share. Douglas said it was too early to project whether the anticipated fourth quarter performance would be “enough for us to exceed the prior year’s overall earnings” of 1.53 dlrs a share.

Figure 1: Example of a portion of a dark photocopy.

Dean Foods Co expects earnings for the fourth quarter ending May 30 to exceed those of the same year-ago period, Chairman Kenneth Douglas told analysts. In the fiscal 1986 fourth quarter the food processor reported earnings of 40 cts a share. Douglas also said the year’s sales should exceed 1.4 billion dlrs, up from 1.27 billion dlrs the prior year. He repeated an earlier projection that third-quarter earnings “will probably be offslightly” from last year’s 40 cts a share, falling in the range of 34 cts to 36 cts a share. Douglas said it was too early to project whether the anticipated fourth quarter performance would be “enough for us to exceed the prior year’s overall earnings” of 1.53 dlrs a share.

Figure 2: OCR output for the image from Figure 1.

better-quality OCR package on noisier inputs of the same type. Figure 1 shows a portion of a dark photocopy page used in our studies, while Figure 2 shows the OCR output from *Tesseract*. Note that the darkening and smearing of character shapes, barely visible to the human eye, leads to various forms of substitution errors (e.g., l → i, h → 11, rn → m) as well as space deletion (e.g., of the → ofthe) and insertion (e.g., project → pro ject) errors.

2.2 Sentence Boundary Detection

Procedures for sentence boundary detection use a variety of syntactic and semantic cues in order to break the input text into sentence-sized units, one per line (i.e., each unit is terminated by a standard end-of-line delimiter such as the Unix newline character). The sentence boundary detector we used in our test is the MXTERMINATOR package by Reynar and Ratnaparkhi [17].

2.3 Tokenization

Tokenization takes the input text which has been divided into one sentence per line and breaks it into individual tokens which are delimited by white space. These largely correspond to word-like units or isolated punctuation symbols. In our studies, we used the Penn Treebank tokenizer [13]. As noted in the documentation, its operation can be summarized as: (1) most punctuation is split from adjoining words, (2) double quotes are changed to doubled single forward- and backward-quotes, and (3) verb contractions and the Anglo-Saxon genitive of nouns are split into their component morphemes, and each morpheme is tagged separately.

2.4 Part-of-Speech Tagging

Part-of-speech tagging takes the tokenized text as input and tags each token as per its part of speech. We used

Ratnaparkhi’s part-of-speech tagger MXPOST [16], which produced a total of 42 different part-of-speech tags for our data.

At this point in the discussion, it is also good for us to note that the evaluations we conduct in this work are *relativistic*. That is, there is no universal ground-truth, but rather we compare the performance of the various text analysis stages on clean and noisy inputs. An “error” is considered to have occurred when the two sets of results differ. There may already in fact be errors present, even for clean inputs.

3. AN EVALUATION PARADIGM

Performance evaluation for text analysis of noisy inputs presents some serious hurdles. The approach we described in our earlier papers makes use of approximate string matching to align two linear streams of text, one representing OCR output and the other representing the ground-truth [10, 11]. Due to the markup conventions employed by sentence boundary detection, tokenization, and part-of-speech tagging, this task is significantly more complex than computing the basic alignments used for assessing OCR accuracy. The nature of the problem is depicted in Figure 3, where there are four sentences detected in the original text and nine in the associated OCR output from a dark photocopy. The challenge, then, is to determine the proper correspondence between purported sentences, tokens, and part-of-speech tags so that errors may be identified and attributed to their root causes.

Still, we can build on the same paradigm used in OCR error analysis [3], employing an optimization framework that likewise can be solved using dynamic programming. We begin by letting $S = s_1 s_2 \dots s_m$ be the source document (the ground-truth), $T = t_1 t_2 \dots t_n$ be the target document (the OCR output), and defining $dist1_{i,j}$ to be the distance between the first i symbols of S and the first j symbols of T . The initial conditions are:

$$\begin{aligned} dist1_{0,0} &= 0 \\ dist1_{i,0} &= dist1_{i-1,0} + c1_{del}(s_i) \\ dist1_{0,j} &= dist1_{0,j-1} + c1_{ins}(t_j) \end{aligned} \quad (1)$$

and the main dynamic programming recurrence is:

$$dist1_{i,j} = \min \begin{cases} dist1_{i-1,j} + c1_{del}(s_i) \\ dist1_{i,j-1} + c1_{ins}(t_j) \\ dist1_{i-1,j-1} + c1_{sub}(s_i, t_j) \end{cases} \quad (2)$$

for $1 \leq i \leq m$, $1 \leq j \leq n$. Here deletions, insertions, and mismatches are charged positive costs, and exact matches are charged negative costs. The computation builds a matrix of distance values working from the upper left corner ($dist1_{0,0}$) to the lower right ($dist1_{m,n}$).

By maintaining the decision(s) used to obtain the minimum in each step, it becomes possible to backtrack the computation and obtain, in essence, an explanation of the errors that arose in processing the input. This information is used in analyzing the performance of the procedure under study.

To generalize these ideas to later stages of text processing, consider the output of those stages and the errors that might arise. Tokenization, for example, might fail to recognize a token boundary thereby combining two tokens into one (a “merge”), or break a token into two more or pieces (a “split”). Similar errors may arise in sentence boundary detection.

In the paradigm we have developed, we adopt a three level hierarchy. At the highest level, sentences (or purported sentences) are matched allowing for missed or spurious sentence boundaries. The basic entity in this case is a sentence string, and the costs of deleting, inserting, substituting, splitting, or merging sentence strings are defined recursively in terms of the next level of the hierarchy, which is tokens. As with the sentence level, tokens can be split or merged. Comparison of tokens is defined in terms of the lowest level of the hierarchy, which is the basic approximate string matching model we began this section with (Equations 1 and 2).

In terms of dynamic programming, at the token level, the algorithm becomes:

$$dist2_{i,j} = \min \begin{cases} dist2_{i-1,j} + c2_{del}(s_i) \\ dist2_{i,j-1} + c2_{ins}(t_j) \\ \min_{1 \leq k' \leq k, 1 \leq l' \leq l} [dist2_{i-k',j-l'} + c2_{sub_{k,l}}(s_{i-k'+1\dots i}, t_{j-l'+1\dots j})] \end{cases} \quad (3)$$

where the inputs are assumed to be sentences and c_{del} , c_{ins} , and c_{sub} are now the costs of deleting, inserting, and substituting whole tokens, respectively, which can be naturally defined in terms of the first-level computation.

Lastly, at the highest level, the input is a whole page and the basic editing entities are sentences. For the recurrence, we have:

$$dist3_{i,j} = \min \begin{cases} dist3_{i-1,j} + c3_{del}(s_i) \\ dist3_{i,j-1} + c3_{ins}(t_j) \\ \min_{1 \leq k' \leq k, 1 \leq l' \leq l} [dist3_{i-k',j-l'} + c3_{sub_{k,l}}(s_{i-k'+1\dots i}, t_{j-l'+1\dots j})] \end{cases} \quad (4)$$

with costs defined in terms of the second-level computation.

By executing this hierarchical dynamic programming from the top down, given an input page for the OCR results as processed through the text analysis pipeline and another page for the corresponding ground-truth, we can determine an optimal alignment between purported sentences, which is defined in terms of an optimal alignment between individual tokens in the sentences, which is defined in terms of an optimal alignment between each possible pairing of tokens (including the possibilities that tokens are deleted, inserted, split, or merged). Once an alignment is constructed using the orthography of the input text strings, we may compare the part-of-speech tags assigned to corresponding tokens to study the impact of OCR errors on that process as well.

From a pragmatic standpoint, this optimization process can require a substantial amount of CPU time depending on the length of the input documents (we have observed run-times of several minutes for pages containing $\pm 1,000$ characters). There are, however, well-known techniques for speeding up dynamic programming (*e.g.*, so-called “beam search”) which have little or no effect on the optimality of the results for the cases of interest.

4. TABLE SPOTTING IN TEXT

As a practical matter, the NLP routines we have described are intended for application to sentential text. However, some collections, including the well-known Reuters-21578 news corpus [9], contain samples that are primarily tabular. Attempting to parse such documents could introduce misleading results in the sorts of studies we have in mind. An example is shown in Figure 4.

Our past work on medium-independent table detection [7] can be applied to identify pages containing tables so that

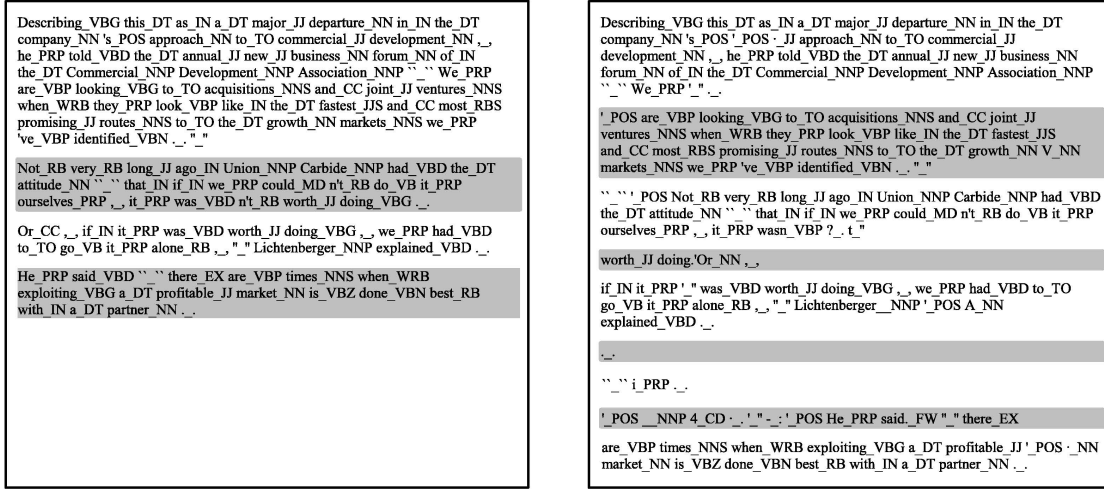


Figure 3: NLP results for an original text (left) and OCR output from a dark photocopy (right).

Average interest rates on yen certificates of deposit, CD, fell to 4.27 pct in the week ended February 25 from 4.32 pct the previous week, the Bank of Japan said. New rates (previous in brackets), were -

Average CD rates all banks	4.27 pct (4.32)
Money Market Certificate, MMC, ceiling rates for the week starting from March 2	3.52 pct (3.57)
Average CD rates of city, trust and long-term banks	
Less than 60 days	4.33 pct (4.32)
60-90 days	4.13 pct (4.37)
Average CD rates of city, trust and long-term banks	
90-120 days	4.35 pct (4.30)
120-150 days	4.38 pct (4.29)
150-180 days	unquoted (unquoted)
180-270 days	3.67 pct (unquoted)
Over 270 days	4.01 pct (unquoted)
Average yen bankers' acceptance rates of city, trust and long-term banks	
30 to less than 60 days	unquoted (4.13)
60-90 days	unquoted (unquoted)
90-120 days	unquoted (unquoted)

Figure 4: Example of a tabular document from the Reuters-21578 news corpus.

they can be held out from the dataset. This paradigm consists of a high-level framework that formulates table detection as an optimization problem along with specific table quality measures that can be tuned for a given application and/or the input medium. We assume that the input is a single column document segmentable into individual, non-overlapping text lines (referred to simply as “lines” henceforth). This assumption is not too restrictive since multi-column input documents can be first segmented into individual columns before running our table detection algorithm.

To preserve generality, it is best not to make any *a priori* assumptions about where table(s) might begin or end in the input (recall Figure 4). Instead, the method we have developed computes a value for all possible starting and ending positions and then chooses the best possible way to partition the input into some number of tables.

Say there are a total of n lines in the input, and let $tab[i, j]$ be a measure of our confidence when lines i through j are interpreted as a single table. Let $merit_{pre}(i, [i + 1, j])$ be the merit of prepending line i to the table extending from line $i + 1$ to line j , and $merit_{app}([i, j - 1], j)$ be the merit of appending line j to the table extending from line i to line $j - 1$. Specific functions for $merit_{pre}$ and $merit_{app}$ typi-

cally return larger values for more compatible combinations. Then we define:

$$tab[i, i] = 0 \quad 1 \leq i \leq n \quad (5)$$

and

$$tab[i, j] = \max \left\{ \begin{array}{l} merit_{pre}(i, [i + 1, j]) + tab[i + 1, j] \\ tab[i, j - 1] + merit_{app}([i, j - 1], j) \end{array} \right\} \quad (6)$$

for $1 \leq i < j \leq n$. This computation builds an upper triangular matrix holding the values for all possible table starting and ending positions and requires time $O(n^2)$.

The best (*i.e.*, highest quality) table in the input can then be found by searching the $tab[i, j]$ matrix for its maximum value. Say this occurs at index $[a, b]$. This means the best table extends from line a to line b . The second-best table can then be located by excluding the region $[a, b]$ and searching the regions $[1, a - 1]$ and $[b + 1, n]$ for the next highest value. This process can be repeated to find the k best tables.

A greedy approach like this could, however, lead to an unsatisfactory solution when the document contains more than one table. This situation can be remedied by formulating the partitioning of the input into tables as an optimization problem. Let $score[i, j]$ correspond to the best way to interpret lines i through j as some number of (*i.e.*, zero or more) tables. The computation is defined as follows:

$$score[i, i] = tab[i, i] \quad 1 \leq i \leq n \quad (7)$$

and

$$score[i, j] = \max \left\{ \begin{array}{l} tab[i, j] \quad 1 \leq i < j \leq n \\ \max_{i \leq k < j} \{ score[i, k] + score[k + 1, j] \} \end{array} \right\} \quad (8)$$

One way to interpret Equation 8 is that the best way to partition lines i through j into some number of tables is to either (1) treat lines i through j as coming from a single table, or (2) break the region into two subregions between lines k and $k + 1$ and consider each separately. Note that if region $[i, j]$ contains exactly one table (or a portion thereof), Case (1) will apply. Otherwise, there must surely exist a k such that the region can be broken into two independent subregions (without splitting a table) and then Case (2) applies.

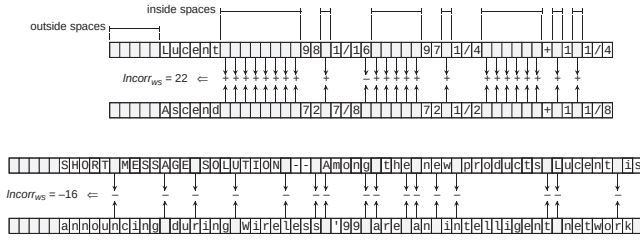


Figure 5: White space correlations for text lines.

The precise decomposition can be obtained by backtracking the sequence of decisions made in evaluating Equation 8. Any region on the optimal path whose *tab* value is higher than a predetermined threshold is considered a table region. Because of the two levels of optimization, algorithm *score* requires time $O(n^3)$.

One of the simplest quality measures imaginable is the degree to which the white space in the text line to be added correlates to the inter-column spacing in the presumed table. On a given line, we distinguish between *inside spaces* and *outside spaces*; the former have at least one non-space character to the left and to the right, while the latter have no non-space characters on one or both sides.

In the case of text input such as the original Reuters news articles, let α and β correspond to specific character positions on two lines. We define:

$$acorr(\alpha, \beta) = \begin{cases} 1 & \text{if } \alpha \text{ and } \beta \text{ are both inside spaces} \\ -1 & \text{if one of } \alpha \text{ or } \beta \text{ is an inside space} \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

Let $atext[i, k]$ be the input text where the first index designates the line and the second the character on that line. For two lines i and j , we pad the shorter line with (outside) spaces to match the length of the longer line. Let this length be m . Now we define our line correlation measure:

$$Incorr_{ws}(i, j) = \sum_{k=1}^m acorr(atext[i, k], atext[j, k]) \quad (10)$$

This is a measure of the correlation between the white space “streams” on the two lines in question. A positive value indicates the lines correlate well (*i.e.*, they may potentially belong to the same table). Figure 5 illustrates this for two text lines that correlate well (top) and badly (bottom).

We now define the set of merit functions:

$$merit_{pre}(i, [i+1, j]) = \sum_{k=i+1}^j \frac{1}{e^{\gamma(k-i-1)}} \cdot Incorr_{ws}(i, k) \quad (11)$$

and

$$merit_{app}([i, j-1], j) = \sum_{k=i}^{j-1} \frac{1}{e^{\gamma(j-1-k)}} \cdot Incorr_{ws}(k, j) \quad (12)$$

where γ is a constant that determines the exponential decay. This gives one possible criterion for judging the quality of text tables.

When run on Split-000 of the Reuters dataset with an aggressive threshold, we determined our approach to table spotting exhibited a recall of 89% (tabular documents that were correctly excluded from the dataset) and an estimated precision of 100% (documents included in the dataset that

were indeed non-tabular). We close by noting there are other reasons why a given text might not be parsable; for example, in the Reuters corpus there are boilerplate reports of changes in commodity prices that, while not tabular, are not sentential either. Still, the net result of this pre-processing step is to yield a subset more appropriate to our purposes.

5. EXPERIMENTAL EVALUATION

As suggested previously, our baseline dataset is derived from “Split-000” of the Reuters-21578 news corpus [9]. After filtering out articles that consist primarily of tabular data, we formatted each of the remaining documents as a single page typeset in Times-Roman 12-point font. In doing so, we discarded articles that were either too long to fit on a page or too short to provide a good test case (fewer than 50 words).

Of the 925 articles in the original set, 661 remained after these various criteria were applied. These pages were then printed on a Ricoh Aficio digital photocopier and scanned back in using the same machine at a resolution of 300 dpi. One set of pages was scanned as-is, another two sets were first photocopied through one and two generations with the contrast set to the darkest possible setting, and two more sets were similarly photocopied through one and two generations at the lightest possible setting before scanning. This resulted in a test set totaling 3,305 pages. We then ran the resulting bitmap images through the *Tesseract* OCR package. Examples of a region of a scanned page image and the associated OCR output were shown in Figures 1 and 2.

Basic OCR accuracy can be judged using a single level of dynamic programming, *i.e.*, Equations 1 and 2, as described elsewhere [3]. These results for the four datasets are presented in Table 1. As in the information retrieval domain, precision and recall are used here to reflect two different aspects of system performance. The former is the fraction of reported entities that are true, while the latter is the fraction of true entities that are reported. Note that the baseline OCR accuracy is quite high, but performance deteriorates for the degraded documents. It is also instructive to consider separately the impact on punctuation symbols and whitespace; these results are also shown in the table. Punctuation symbols in particular are badly impacted, with a large number of false alarms (low precision), especially in the case of the Dark2 dataset where fewer than 80% of the reports are true. This phenomenon has serious implications for sentence boundary detection and later stages of text processing.

We then ran sentence boundary detection, tokenization, and part-of-speech tagging on both the original (ground-truth) news stories and the versions that had been OCR’ed, comparing the results using the paradigm described earlier. This allowed us to both quantify performance as well as to determine the optimal alignments between sequences and hence identify the actual errors that had arisen.

An example of a relatively straightforward alignment produced by our evaluation procedure is shown in Figure 6. This displays the effect of a single-character substitution error (i being misrecognized as ;). The result is three tokens where before there was only one. Not unexpectedly, two of the three tokens have inappropriate part-of-speech labels. In this instance, the OCR error impacts two text analysis stages and is, for the most part, localized; other errors can

DT	NN	VBD	PRP	VBD	VBG	DT	NNS	IN	DT	NN	.
The	company	said	it	was	receiving	no	proceeds	from	the	offering	.

Ground-Truth

DT	NN	VBD	PRP	VBD	NN	:	VBG	DT	NNS	IN	DT	NN	.
The	company	said	it	was	rece	:	ving	no	proceeds	from	the	offering	.

OCR Output

Figure 6: Example of an alignment displaying impact of a single substitution error.

Table 1: Average OCR performance relative to ground-truth.

	All Symbols			Punctuation			Whitespace		
	Prec.	Recall	Overall	Prec.	Recall	Overall	Prec.	Recall	Overall
Clean	0.995	0.997	0.997	0.981	0.996	0.988	0.995	0.999	0.997
Dark1	0.989	0.996	0.994	0.937	0.992	0.963	0.980	0.998	0.989
Dark2	0.966	0.990	0.981	0.797	0.972	0.874	0.929	0.988	0.958
Light1	0.995	0.997	0.997	0.977	0.994	0.986	0.993	0.999	0.996
Light2	0.994	0.997	0.997	0.971	0.989	0.981	0.992	0.999	0.996
Overall	0.988	0.995	0.993	0.933	0.989	0.958	0.978	0.997	0.987

have effects that cascade through the pipeline, becoming amplified at each stage.

A tabulation of the dynamic programming results for the three text processing stages appears in Table 2. While most of the data is processed with relatively high accuracy, the computed rates are generally lower than the input OCR accuracies. Note, for example, that the overall OCR accuracy across all symbol classes is 99.3%, whereas the sentence boundary detection, tokenization, and part-of-speech tagging accuracies are 94.7%, 97.9%, and 96.6%, respectively. Recall also that these measures are relative to the exact same procedures run on the original (error-free) text. This illustrates the cascading effect of OCR errors in the text analysis pipeline. The dark photocopied documents show particularly poor results, undoubtedly because of the large number of spurious punctuation symbols they introduce. Here we see that character recognition errors can have, at times, a relatively large impact on one or more of the downstream NLP stages. Sentence boundary detection appears particularly susceptible in the worst case.

In addition to analyzing accuracy rates, it is also instructive to consider counts of the average number of errors that occur on each page. This data is presented in Table 3, broken down by error type for each of the NLP pipeline stages. While always altering the input text, in the best case an OCR error might result in no mistakes in sentence boundary detection, tokenization, or part-of-speech tagging: those procedures could be agnostic (or robust) to the error in question. Here, however, we can see that on average, each page contains a number of induced tokenization and part-of-speech tagging errors, and sentence boundary detection errors also occur with some regularity.

Errors that arise in later stages of processing may be due to the original OCR error, or to an error it induced in an earlier pipeline stage. Whatever the cause, this error cascade is an important artifact of pipelined text analysis systems. In Figures 7-9, we plot the accuracy for each of the three NLP stages as a function of the input OCR accuracy for all 3,305 documents in our dataset. In viewing the charts, note that the x-axis (OCR accuracy) ranges from 90% to 100%, whereas the range for the y-axis is from 0% to 100%. Accuracy of the NLP stages is nearly always uniformly lower

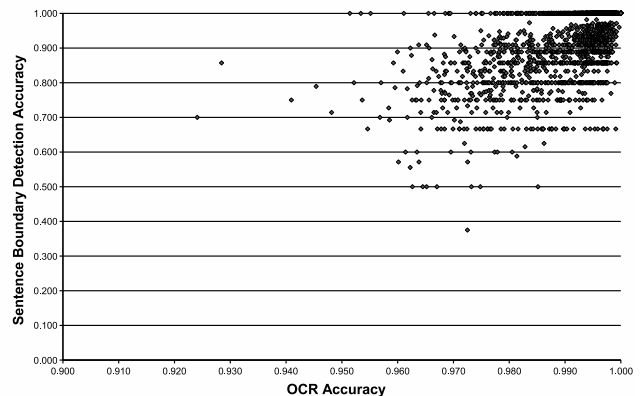


Figure 7: Sentence boundary detection accuracy as a function of OCR accuracy.

than the OCR accuracy, sometimes substantially so.

6. CONCLUSIONS

In this paper, we considered a text analysis pipeline consisting of four stages: optical character recognition, sentence boundary detection, tokenization, and part-of-speech tagging. Using a formal algorithmic model for evaluating the performance of multi-stage processes, we presented experimental results examining the impact of representative OCR errors on later stages in the pipeline. While most such errors are localized, in the worst case some have an amplifying effect that extends well beyond the site of the original error, thereby degrading the performance of the end-to-end system. Studies such as this provide a basis for the development of more robust text analysis techniques, as well as guidance for tuning OCR systems to achieve optimal performance when embedded in larger applications.

Since errors propagate from one stage of the pipeline to the next, sentence boundary detection algorithms that work reliably for noisy documents are clearly important. Similarly, the majority of errors that occurred in our study are

Table 2: Average NLP performance relative to ground-truth.

	Sentence Boundaries			Tokenization			Part-of-Speech Tagging		
	Prec.	Recall	Overall	Prec.	Recall	Overall	Prec.	Recall	Overall
Clean	0.978	0.995	0.985	0.994	0.997	0.995	0.988	0.991	0.989
Dark1	0.918	0.988	0.946	0.977	0.987	0.982	0.964	0.976	0.970
Dark2	0.782	0.963	0.850	0.919	0.946	0.932	0.885	0.917	0.900
Light1	0.971	0.994	0.981	0.992	0.996	0.994	0.985	0.989	0.987
Light2	0.967	0.984	0.972	0.990	0.994	0.992	0.983	0.987	0.985
Overall	0.923	0.985	0.947	0.974	0.984	0.979	0.961	0.972	0.966

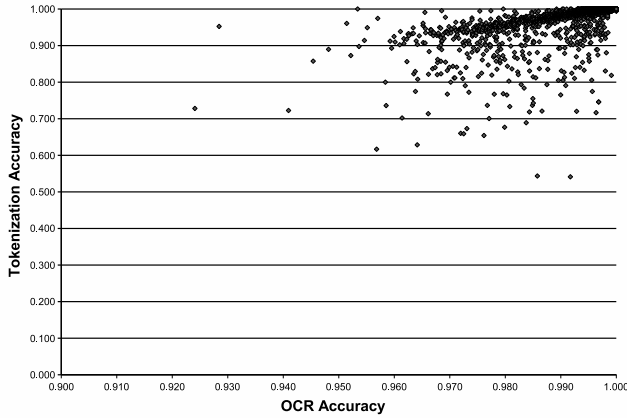


Figure 8: Tokenization accuracy as a function of OCR accuracy.

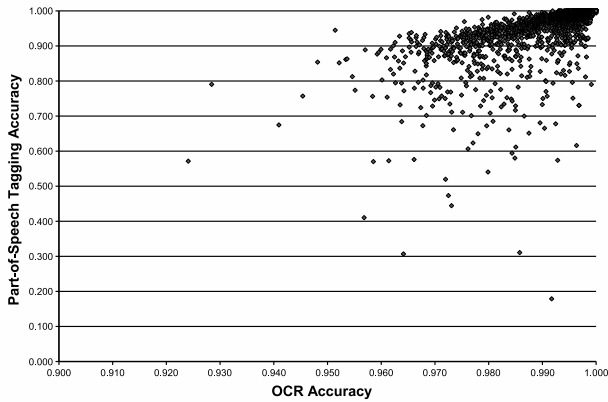


Figure 9: Part-of-speech tagging accuracy as a function of OCR accuracy.

Table 3: Average NLP error counts per document.

	Sentence Boundaries		Tokenizations		Part-of-Speech Tagging Mismatches
	Missed	Added	Missed	Added	
Clean	0.0	0.1	0.5	1.0	0.8
Dark1	0.1	0.5	2.2	3.8	1.5
Dark2	0.2	1.6	8.7	12.7	4.3
Light1	0.0	0.2	0.9	1.4	0.9
Light2	0.1	0.2	1.0	1.7	0.9
Overall	0.1	0.5	2.7	4.1	1.7

tokenization or part-of-speech tagging errors which would feed into additional text processing routines in a real system, contributing to a further error cascade. One possible approach for attempting to address this issue would be to retrain existing systems on such documents to make them more tolerant of noise. This line of attack would be analogous to techniques now being developed to improve NLP performance on informal and/or ungrammatical text [5, 20]. However, this is likely to be effective only when noise levels are relatively low. Much work needs to be done to develop robust methods that can handle documents with high noise levels.

In the context of document summarization, a potential downstream application, we have previously noted that the quality of text analysis is directly tied to the level of noise in a document [8]. Summaries are not seriously impacted in the presence of minor errors, but as errors increase, the results may range from being difficult to read, to incomprehensible. Here it would be useful to develop methods for assessing noise levels in an input image without requiring access to ground-truth. Such measurements could be incorporated into text analysis algorithms for the purpose of segmenting out problematic regions of the page for special processing (or even avoiding them entirely), thereby improving overall readability. Past work on attempting to quantify document image quality for predicting OCR accuracy [1, 2, 6] addresses a related problem, but one which exhibits some notable differences. Establishing a robust index that measures whether a given section of text can be processed reliably is one possible approach.

We also observe that our current study, while employing real data generated from a large collection of scanned documents, is still limited in that the page layouts, textual content, and image degradations are all somewhat simplistic. This raises interesting questions for future research concerning the interactions between OCR errors that might occur in close proximity, as well as higher-level document analysis errors that can impact larger regions of the page. Are such errors further amplified downstream? Is the cumulative effect more additive or multiplicative? The answers to questions such as these will prove important as we seek to build more sophisticated systems capable of handling real-world document processing tasks for inputs that range both widely in content and quality.

Finally, we conclude by noting that datasets designed for studying problems such as the ones described in this paper can be an invaluable resource to the international research community. Hence, we are making our large collection of scanned pages and the associated ground-truth and intermediate analyses available online [12].

7. ACKNOWLEDGMENTS

We gratefully acknowledge support from the National Science Foundation under Award CNS-0430178 and a DARPA IPTO grant administered by BBN Technologies. We also thank the anonymous reviewers for their helpful comments.

8. REFERENCES

- [1] L. R. Blando, J. Kanai, and T. A. Nartker. Prediction of OCR accuracy using simple image features. In *Proceedings of the Third International Conference on Document Analysis and Recognition*, pages 319–322, Montréal, Canada, August 1995.
- [2] M. Cannon, J. Hochberg, and P. Kelly. Quality assessment and restoration of typewritten document images. Technical Report LA-UR 99-1233, Los Alamos National Laboratory, 1999.
- [3] J. Esakov, D. P. Lopresti, J. S. Sandberg, and J. Zhou. Issues in automatic OCR error classification. In *Proceedings of the Third Annual Symposium on Document Analysis and Information Retrieval*, pages 401–412, April 1994.
- [4] F. Farooq and Y. Al-Onaizan. Effect of degraded input on statistical machine translation. In *Proceedings of the Symposium on Document Image Understanding Technology*, pages 103–109, November 2005.
- [5] J. Foster. Treebanks gone bad: Generating a treebank of ungrammatical English. In *Proceedings of the Workshop on Analytics for Noisy Unstructured Text Data*, Hyderabad, India, January 2007.
- [6] V. Govindaraju and S. N. Srihari. Assessment of image quality to predict readability of documents. In *Proceedings of Document Recognition III (IS&T/SPIE Electronic Imaging)*, volume 2660, pages 333–342, San Jose, CA, January 1996.
- [7] J. Hu, R. Kashi, D. Lopresti, and G. Wilfong. Medium-independent table detection. In D. P. Lopresti and J. Zhou, editors, *Proceedings of Document Recognition and Retrieval VII (IS&T/SPIE Electronic Imaging)*, volume 3967, pages 291–302, San Jose, CA, January 2000.
- [8] H. Jing, D. Lopresti, and C. Shih. Summarizing noisy documents. In *Proceedings of the Symposium on Document Image Understanding Technology*, pages 111–119, April 2003.
- [9] D. D. Lewis. Reuters-21578 Test Collection, Distribution 1.0, May 2008.
<http://www.daviddlewis.com/resources/testcollections/reuters21578/>.
- [10] D. Lopresti. Performance evaluation for text processing of noisy inputs. In *Proceedings of the 20th Annual ACM Symposium on Applied Computing (Document Engineering Track)*, pages 759–763, Santa Fe, NM, March 2005.
- [11] D. Lopresti. Measuring the impact of character recognition errors on downstream text analysis. In *Proceedings of Document Recognition and Retrieval XV (IS&T/SPIE Electronic Imaging)*, volume 6815, pages 0G.01–0G.11, San Jose, CA, January 2008.
- [12] D. Lopresti. Noisy OCR text dataset, May 2008.
<http://www.cse.lehigh.edu/~lopresti/noisytext.html>.
- [13] R. MacIntyre. Penn Treebank tokenizer (sed script source code), 1995.
<http://www.cis.upenn.edu/~treebank/tokenizer.sed>.
- [14] D. Miller, S. Boisen, R. Schwartz, R. Stone, and R. Weischedel. Named entity extraction from noisy input: Speech and OCR. In *Proceedings of the 6th Applied Natural Language Processing Conference*, pages 316–324, Seattle, WA, 2000.
- [15] D. D. Palmer and M. Ostendorf. Improving information extraction by modeling errors in speech recognizer output. In J. Allan, editor, *Proceedings of the First International Conference on Human Language Technology Research*, 2001.
- [16] A. Ratnaparkhi. A maximum entropy part-of-speech tagger. In *Proceedings of the Empirical Methods in Natural Language Processing Conference*, May 1996.
<ftp://ftp.cis.upenn.edu/pub/adwait/jmx/jmx.tar.gz>.
- [17] J. C. Reynar and A. Ratnaparkhi. A maximum entropy approach to identifying sentence boundaries. In *Proceedings of the Fifth Conference on Applied Natural Language Processing*, Washington, DC, March–April 1997.
<ftp://ftp.cis.upenn.edu/pub/adwait/jmx/jmx.tar.gz>.
- [18] K. Taghva, J. Borsack, and A. Condit. Effects of OCR errors on ranking and feedback using the vector space model. *Information Processing and Management*, 32(3):317–327, 1996.
- [19] Tesseract open source OCR engine, May 2008.
<http://code.google.com/p/tesseract-ocr/>.
- [20] *Workshop on Analytics for Noisy Unstructured Text Data*. Hyderabad, India, January 2007.
<http://research.ihost.com/and2007/>.